Product Inventory Management System Backend Project Documentation

Team No: 9

Team Head:Loshini.P

(E44DE8E542970E2B99A48E647ECEDC4D)

Team Member: Kowsika.R

(37EC17AF63C33EDA92542F4328ED8443)

Team Member: Meha Sona.S.M

(EFA3E19843360C5A31A46E172DF4EC3E)

Team Member: Mitha.R

(03F8588396EC00FA23143570916E7090)

Project Overview:

The Product Inventory Management System is a backend web application designed to manage product data securely and efficiently. The system is built using modern backend technologies such as Node.js, Express.js, and MongoDB. It follows RESTful API design principles and provides secure authentication using JSON Web Tokens (JWT). The main purpose of this system is to allow users to manage their product inventory through a structured backend architecture.

Becomes difficult and time-consuming. This project solves that problem by providing a digital platform where users can register, log in, and manage their product data. The system ensures that only authenticated users can access their own product information, making it secure and reliable.

The application focuses mainly on backend development concepts such as authentication, middleware, database schema design, and RESTful

API implementation. It also demonstrates the use of scalable architecture that can be expanded into a full-stack application in the future.

Objectives of the Project:

The main objective of this project is to develop a secure and scalable backend application for product inventory management. The system uses JWT-based authentication to ensure that only authorized users can access protected APIs.

Another important objective is to provide complete CRUD operations. CRUD stands for Create, Read, Update, and Delete. Users should be able to add new products, view their existing products, update product information such as price or quantity, and delete products when they are no longer needed.

The project also aims to implement a structured backend architecture using Node.js and Express.js. By organizing the project into controllers, routes, models, and middleware, the system becomes easier to maintain and scale. Additionally, the project demonstrates database integration using MongoDB and Mongoose. MongoDB stores the product and user information while Mongoose helps define schemas and manage data efficiently.

Target Users:

This system is useful for several types of users. Small business owners can use the system to track their inventory and manage product records. Instead of maintaining product data manually, they can store everything in a secure database

Freelancers and online sellers can also use the system to track items or services they provide. For example, someone selling products online can maintain product names, prices, quantities, and descriptions using the system.

Students who are learning backend development can use this project as a reference or learning model. The system demonstrates important backend concepts such as authentication, API development, database connection, and middleware usage.

Developers who are exploring Node.js and Express.js can also benefit from studying this architecture. The project provides a practical example of how backend services are structured in modern web applications.

Project Instructions:

To develop the backend project, the first step is to create a project folder. Inside the folder, the Node.js environment needs to be initialized using the npm initialization command. This command generates the package.json file which manages all project dependencies.

Next, an Express.js server must be configured. Express is a lightweight backend framework used to create APIs and manage server logic. The Express server listens on a specific port and handles incoming client requests.

After configuring the server, MongoDB database connectivity must be established. MongoDB is used to store application data such as user information and product records. The connection is created using Mongoose. Once the database connection is ready, authentication APIs must be implemented. These APIs allow users to register and log in to the system. Passwords are hashed before storing them in the database to maintain security.

Finally, product APIs must be implemented to perform CRUD operations. These APIs allow users to manage product data. After development is completed, the APIs can be tested using tools such as Thunder Client inside Visual Studio Code.

Pre-Requisites:

Before starting the project, several software tools must be installed on the system. Node.js is required to run JavaScript on the server side. It provides the runtime environment needed for backend development.

MongoDB must also be installed or accessed through MongoDB Atlas. This database stores the application's data in a document-based format. MongoDB works well with Node.js and provides flexible data storage.

The npm package manager is used to install and manage dependencies such as Express, Mongoose, and JWT libraries. Visual Studio Code is recommended as the development environment. It provides useful extensions, debugging tools, and built-in terminal support which helps developers write and test code efficiently.

Project Architecture:

The project follows a three-tier backend architecture. The first layer is the client layer, which sends requests to the server. The second layer is the backend server built using Node.js and Express.js. The third layer is the database layer where data is stored using MongoDB.

The backend server acts as the central component that processes requests, verifies authentication tokens, and communicates with the database. Each request goes through middleware for validation and authentication before reaching the controller.

Controllers contain the business logic of the application. They interact with database models to retrieve or update data. Routes are responsible for mapping URLs to specific controller functions. This structured architecture improves code organization and makes the system easier to maintain, debug, and expand in the future.

Database Architecture:

The system uses MongoDB as the primary database. MongoDB is a NoSQL document-based database which stores data in JSON-like documents. This structure makes it flexible and easy to work with when building web applications. Two main collections are used in this project: the Users collection and the Products collection

The Users collection stores information such as the user's name, email, and hashed password. Each record also contains timestamps to track creation and update times. The Products collection stores product-related information such as product name, price, quantity, description, and a reference to the user who created the product.

This reference ensures that each product belongs to a specific user. The relationship between the two collections is one-to-many. This means that one user can have many products in their inventory.

Backend Development:

Backend development in this project is performed using Node.js and Express.js. Node.js provides the runtime environment while Express simplifies API creation and request handling.

Controllers manage the main application logic. For example, the authentication controller handles user registration and login. It validates input data, hashes passwords using bcrypt, and generates JWT tokens for secure access.

The product controller manages inventory operations. It allows users to create products, view their products, update details, and delete products from the database.

Middleware functions play an important role in security. The authentication middleware verifies JWT tokens before allowing access to protected routes. If a request does not contain a valid token, the system blocks the request and returns an error.

API Testing:

After the backend development is completed, the APIs must be tested to ensure that they work correctly. API testing can be performed using Thunder Client or Postman. The first step is to test the user registration API.

This API allows a new user to create an account by providing their name, email, and password. The system validates the information and stores the user data in the database. Next, the login API must be tested. When a user enters their credentials, the system verifies the information and generates a JWT token. This token is used to access protected endpoints. Product APIs can then be tested.

The create product API allows users to add products to their inventory. The get products API retrieves all products associated with the

logged-in user. The update product API modifies product details, while the delete product API removes products from the database.

Conclusion:

The Product Inventory Management System demonstrates the development of a secure and scalable backend application using Node.js, Express.js, and MongoDB. The project highlights important backend development concepts such as RESTful APIs, authentication using JWT, middleware usage, and database schema design

By implementing user authentication and protected routes, the system ensures that users can manage only their own data. This approach improves security and maintains data privacy. The modular architecture used in this project makes it easy to extend the system in the future. For example, a frontend application can be integrated to create a complete full-stack product management platform.

Overall, this project serves as an excellent learning resource for students and developers who want to understand how backend systems are designed and implemented in modern web development.